

Component Compression and Restart Entropy in Exact Unknot Recognition

Arithmetic improvements and a conditional route to quasi-polynomial complexity

Research contribution prepared for the ProveIt project

7 October 2026

Abstract

We develop two complementary improvements to ProveIt’s unknot-recognition program. First, a compiled genus-zero dotted-cobordism composition factors through a square-zero algebra with one variable per connected gluing component. Ranked subset convolution then replaces dense pairs of input monomials by a $\text{poly}(w)2^{w/2}$ -bit-time composition algorithm at boundary size w . The component quotient is the exact observational quotient for all opposite operands; in particular, it is dimension-optimal among universal linear summaries. An explicit family of planar matching triples realizes arbitrarily large input circle spaces with a fixed number of gluing components. A standard-library Python implementation is cross-checked against an excerpt of the repository’s numerical routines, with 460,564 matching-composition comparisons and 1,000 associativity checks. Paired dense numerical-kernel benchmarks show substantial improvements, while a deliberately sparse case shows why unconditional replacement would be inappropriate.

Second, we refine the combinatorics of lexicographic hierarchy restarts. If length- L complexity profiles have coordinates at most g and at most s positive coordinates, the exact state bound is $\sum_{j \leq s} \binom{L}{j} g^j$, rather than $(g+1)^L$. This yields quasi-polynomial restart counts when L, g are polynomial and $s = O(\log n)$, even without logarithmic total depth. We give an exact rank potential, a sharpness construction, and executable finite-trace checks. The topology needed to establish this sparsity uniformly, and the complete bit-cost bounds for hierarchy construction, remain explicit research obligations. This work does not establish an unrestricted quasi-polynomial unknot recognizer.

Artifact status. Complete proofs of the algebraic and counting statements; tested numerical kernel and trace tools; opt-in integration harness. No full-recognizer speedup, full-repository test run, or first-in-literature priority is asserted.

Contents

1	Scope, baseline, and the two complexity problems	3
1.1	Contributions and their status	3
2	Dotted-cobordism conventions	4
3	Factorization through connected components	5
3.1	Reconstruction without branching over outputs	5
4	Fast exact arithmetic and bit complexity	6
4.1	Disjoint subset convolution	6
4.2	Exact rank lanes in Python integers	6
4.3	Projection, reconstruction, and complexity	7
5	Optimal information compression and explicit planar families	8
5.1	The exact observational quotient	8
5.2	Planar zipper triples	9
6	Integration and the remaining multiplicity problem	10
6.1	What the adapter changes	10
6.2	A parameter ledger for complete scans	10
6.3	Fixed boundary does not control explicit multiplicity	11
7	Sparse hierarchy restart entropy	12
7.1	The source of the lexicographic profile	12
7.2	An exact executable rank potential	13
7.3	Other entropy budgets	13
7.4	What would complete a hierarchy-based quasi-polynomial proof	14
8	Validation and performance experiments	14
8.1	What was executed	14
8.2	Paired numerical-kernel measurements	15
8.3	Reproduction and repository validation	16
9	Further research questions and completion criteria	16
9.1	Production relevance and adaptive arithmetic	16
9.2	Representations that do not enumerate multiplicity	17
9.3	Low-entropy hierarchy construction	17
9.4	Verification and scan scheduling	18
10	Conclusion	18
A	Artifact map and audit boundaries	19

1 Scope, baseline, and the two complexity problems

The practical objective is a reliable decision procedure for a knot diagram, with an eventual target of $n^{O(\log n)}$ running time. The relevant ProveIt subtree contains an exact implementation and a separate, unfinished hierarchy-based complexity program. These should be improved together, but not conflated.

The audited Python pipeline first applies diagram simplification and one-sided invariant filters, then uses a scanning Khovanov computation when necessary. Its numerical backend already interns matchings, compiles gluing plans, caches shape-dependent results, and cancels units with a sparse elimination strategy. Visible connected-sum factorization and multiple scan orders are also existing features. The repository describes the complete fallback as exponential, not quasi-polynomial [1].

This article targets a remaining dense-arithmetic cost in `fastunknot/planar.py`: on a numerical composition-cache miss, the implementation iterates over the support of each operand and evaluates every pair. The exact audited blob is `1078526e7e7dbaf0b7267728105d870d94136769`. The scanner interface was checked against blob `2c1ad52d14296b109376af19668ae0eebb4c6fdd` of `scan_fast.py`. These are content identifiers, not a claim to have executed a complete checkout.

There are two different sources of large computations. One is the cost of composing two already represented morphisms. The other is the number of objects and morphisms that the computation must represent. Improving the first does not automatically control the second. Our component-compression theorem addresses the first; the multiplicity example in Section 6.3 identifies a genuine obstruction to using it as a universal width-only theorem.

The hierarchy direction has a different distinction: a bound on the number of restarts is not yet a bound on the cost of finding surfaces, cutting along them, and maintaining their descriptions. We formulate the sparse-profile criterion so that these additional obligations cannot disappear inside an unspecified primitive.

1.1 Contributions and their status

The main unconditional contribution is an exact factorization of the repository’s composition law through its connected gluing components, together with a dense-bit implementation and its correctness proof. We prove that this quotient retains precisely the information that any universal opposite-operand composition query can observe. We also give explicit planar “zipper” triples, rather than relying only on artificial component arrays, to exhibit large compression ratios.

The second unconditional contribution is a sparse lexicographic restart potential. Its count is elementary, but its significance for the research program is concrete: one may seek logarithmically many positive-complexity layers instead of logarithmic total hierarchy length. The conditional algorithmic theorem spells out the constructive topological assumptions needed to exploit that change.

Fast subset convolution itself is not new; our implementation uses the ranked zeta-transform method of Björklund, Husfeldt, Kaski, and Koivisto [4]. Dotted-cobordism and local tangle simplification are likewise standard foundations [2, 3]. The contribution claimed here is the proved application, the exact quotient analysis, the concrete implementation, and the refined research target. A literature-priority claim would require a broader comparison than the targeted source review performed for this article.

2 Dotted-cobordism conventions

All coefficients are in $\mathbb{F}_2$, and all vector spaces in the numerical analysis forget quantum gradings. Homological degrees remain part of the scanner's objects. A universal statement about arbitrary coefficient vectors below should not be silently restricted to one graded subspace.

For a finite set I , put

$$\mathcal{R}(I) = \mathbb{F}_2[x_i : i \in I]/(x_i^2 : i \in I), \quad x^S = \prod_{i \in S} x_i.$$

Write $\mathcal{R}_r = \mathcal{R}([r])$, using either zero-based or one-based labels as long as the choice is consistent. Its vector-space dimension is 2^r . A morphism on r circles is represented by an integer

$$\text{enc}(f) = \sum_{S \subseteq [r]} f_S 2^{\text{mask}(S)}.$$

Thus a subset mask and the position of its coefficient bit are distinct levels of encoding. Addition of morphisms is bitwise XOR. An integer may contain 2^r coefficient bits; operations on it are not unit-cost operations.

The Frobenius algebra on one circle is $A = \mathbb{F}_2[x]/(x^2)$, with

$$\varepsilon(1) = 0, \quad \varepsilon(x) = 1, \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x.$$

These are the usual characteristic-two dotted-cobordism rules [2]. The handle operator $m\Delta$ vanishes: it sends 1 to $2x = 0$ and x to $x^2 = 0$. Consequently a connected component of positive genus contributes zero.

Definition 2.1 (Compiled component plan). A genus-zero plan is a list

$$(L_j, R_j, B_j, e_j), \quad 1 \leq j \leq k,$$

where L_j , R_j , and B_j partition the left input circles, right input circles, and output circles respectively. Empty parts are allowed in the general algebraic statement. The nonnegative integer e_j is the number of extra dots. The three circle universes have sizes r, s, t .

For one pair of input monomials x^U, z^V , component j carries

$$d_j = |U \cap L_j| + |V \cap R_j| + e_j$$

dots. If $d_j \geq 2$, the answer is zero. For $d_j = 1$, it contributes the monomial with all output circles in B_j dotted. For $d_j = 0$ and $B_j \neq \emptyset$, it contributes $\sum_{b \in B_j} x^{B_j \setminus \{b\}}$. For $B_j = \emptyset$ the contribution is the scalar $\varepsilon(x^{d_j})$. The full value is the product of component contributions, extended bilinearly in the inputs. This is precisely the numerical rule used by the source-excerpt reference harness.

A component in `planar.py` has a fifth, redundant field containing the possible undotted-output choices. The new implementation accepts either four or five fields and checks the fifth field for consistency. It also checks disjointness of masks and contiguous circle indices. Geometry and circle-numbering conventions are not recomputed by the replacement kernel.

3 Factorization through connected components

Let $y_1, \dots, y_k$ be variables indexed by the gluing components. Define algebra homomorphisms

$$P_L : \mathcal{R}_r \longrightarrow \mathcal{R}_k, \quad x_i \longmapsto y_j \quad (i \in L_j), \quad P_R : \mathcal{R}_s \longrightarrow \mathcal{R}_k, \quad z_i \longmapsto y_j \quad (i \in R_j).$$

These maps are well defined because the image of each square is zero. Different input variables on one component acquire the same image, so a monomial containing two such variables vanishes. Surviving monomials with the same component subset are added modulo two.

For a finite output block B define a linear map $D_B : A \rightarrow \mathcal{R}(B)$ by

$$\begin{aligned} B \neq \emptyset : \quad D_B(1) &= \sum_{b \in B} x^{B \setminus \{b\}}, \quad D_B(x) = x^B, \\ B = \emptyset : \quad D_B(1) &= 0, \quad D_B(x) = 1. \end{aligned}$$

Set $\Omega = \bigotimes_{j=1}^k D_{B_j}$, with tensor factors ordered by the compiled component list. This is a linear reconstruction map, not generally an algebra homomorphism.

Theorem 3.1 (Component factorization). *If every e_j is zero or one and $E = \{j : e_j = 1\}$, then the compiled composition satisfies*

$$\text{Comp}(f, g) = \Omega\left(y^E P_L(f) P_R(g)\right). \quad (1)$$

If some $e_j \geq 2$, the composition is identically zero. A positive-genus component also makes the composition identically zero over $\mathbb{F}_2$.

Proof. It suffices to take monomial inputs. If two left dots lie on one component, P_L sends the monomial to zero; the same holds on the right. If one left and one right dot lie on the same component, their product contains y_j^2 and is zero. Multiplication by y^E imposes exactly the analogous condition for extra dots. Thus the product survives precisely when every component has at most one dot, and its j th exponent is then the total number of dots on that component. Applying D_{B_j} gives its prescribed component value, including the count when the output block is empty. Multiplying these values gives the reference rule. Bilinearity proves the identity for all inputs. The two vanishing statements follow from $x^2 = 0$ and $m\Delta = 0$. $\square$

The quotient calculation is stronger than merely rejecting incompatible monomial pairs early. Terms can cancel *before* the two operands are multiplied. For example, two single dots on different circles of the same component have equal images and their sum disappears.

3.1 Reconstruction without branching over outputs

Lemma 3.2 (Unique component preimage of an output coefficient). *Fix an output subset $X \subseteq [t]$. For each component with a nonempty output block, count the missing dots $|B_j \setminus X|$. If this count is at least two for any j , the coefficient of x^X in $\Omega(h)$ is zero. Otherwise define $Z(X) \subseteq [k]$ by including j exactly when $B_j = \emptyset$ or $B_j \subseteq X$. Then*

$$[x^X] \Omega(h) = [y^{Z(X)}] h.$$

In particular, in (1) the output coefficient is zero unless $E \subseteq Z(X)$; when this condition holds it is $[y^{Z(X) \setminus E}] P_L(f) P_R(g)$.

Proof. For a nonempty block, $D_B(x)$ has every output circle dotted, whereas each monomial of $D_B(1)$ has exactly one missing dot. These supports are disjoint. For an empty block, only $D_B(x)$ survives. The component input exponent is therefore uniquely determined by X . Tensoring the blocks gives the first formula. Multiplication by y^E kills monomials already meeting E and adjoins E to every surviving monomial, giving the second formula. $\square$

This lemma permits a dense pass over 2^t output coefficients. It avoids materializing a separate expansion of $\Omega(y^S)$ for every component subset S .

Example 3.3 (Cancellation in a two-component endomorphism algebra). Let every component have one left, one right, and one output circle. Then P_L, P_R, Ω are the identity identifications. For

$$f = 1 + x_1 + x_1x_2, \quad g = 1 + x_2,$$

one obtains $fg = 1 + x_1 + x_2$, since the two x_1x_2 terms cancel. With zero-based subset masks, the corresponding integer computation is $\text{Comp}(11, 5) = 7$. This example is included in `results/examples.json`.

4 Fast exact arithmetic and bit complexity

4.1 Disjoint subset convolution

Write $a = P_L(f)$ and $b = P_R(g)$. Multiplication in $\mathcal{R}_k$ is the disjoint subset convolution

$$h(S) = \sum_{T \subseteq S} a(T)b(S \setminus T), \quad S \subseteq [k]. \quad (2)$$

The direct implementation uses 3^k scalar products. More importantly for the repository, multiplying before projection can use as many as 2^{r+s} input-monomial pairs. Projection must therefore precede convolution.

For completeness, we give the ranked-transform argument underlying the classical $O(k^2 2^k)$ ring-operation bound [4]. Define

$$a_i(S) = \sum_{\substack{T \subseteq S \\ |T|=i}} a(T), \quad b_i(S) = \sum_{\substack{T \subseteq S \\ |T|=i}} b(T), \quad c_\ell(S) = \sum_{i+j=\ell} a_i(S)b_j(S).$$

Apply subset Möbius inversion to c_ℓ . A term indexed by input subsets U, V remains at S exactly when $U \cup V = S$. Selecting $\ell = |S|$ also requires $|U| + |V| = |S|$, hence $U \cap V = \emptyset$. The selected coefficient is (2). In characteristic two, the subset Möbius transform is the same XOR butterfly as the zeta transform.

4.2 Exact rank lanes in Python integers

The implementation packs ranks into separated binary lanes. Put

$$\ell = \lfloor \log_2(k+1) \rfloor + 1, \quad B = 2^\ell > k+1, \quad A(S) = \sum_{i=0}^k a_i(S)B^i.$$

The initial arrays contain $a(S)B^{|S|}$ and $b(S)B^{|S|}$. XOR zeta butterflies produce the packed arrays of rank parities. Each rank digit is zero or one. In the ordinary integer product $A(S)B'(S)$, a coefficient of any power of the lane base is at most $k + 1$. Thus there is no carry between lanes. Keeping the low bit of each lane computes rank-polynomial multiplication modulo two exactly. The subsequent XOR Möbius transform and extraction of lane $|S|$ produce $h(S)$.

This is integer arithmetic throughout. There is no floating-point convolution, randomized polynomial identity test, or probabilistic error. The use of a lane base *strictly* exceeding $k + 1$ matters; treating an ordinary integer product as carryless without this separation would be incorrect.

4.3 Projection, reconstruction, and complexity

For a left monomial mask $U \neq 0$, remove its least set bit i . If the projection of $U \setminus \{i\}$ is already zero, or already contains the component containing i , record zero. Otherwise adjoin that component. This constructs a projection table of length 2^r with one recurrence per entry. The right table is analogous. The output table uses Lemma 3.2. These tables can be reused for a fixed topological plan.

Let $M(q)$ denote an upper bound for multiplying q -bit integers, and let $W = (k + 1) \lceil \log_2(k + 2) \rceil$. A direct accounting of the packed transforms gives the following statement.

Theorem 4.1 (Dense composition bound). *A genus-zero plan with dimensions (r, s, t, k) can be compiled and evaluated exactly with bit cost bounded by*

$$\text{poly}(r + s + t + k) (2^r + 2^s + 2^t) + O\left(2^k(kW + M(W))\right). \quad (3)$$

In particular, its bit cost is $\text{poly}(r + s + t + k)2^{\max\{r, s, t, k\}}$. Space has the same exponential parameter, up to polynomial factors.

Proof. The two projection tables store one $O(k)$ -bit component mask, or a failure marker, per input subset. Building them costs a polynomial number of bit operations per entry. Reading a packed input as bytes and accumulating surviving terms takes at most polynomial overhead per coefficient position. The output table is built by scanning the k blocks for each of 2^t subsets, and reconstruction is one coefficient lookup per output subset.

The ranked arrays have 2^k entries of $O(W)$ bits. There are $O(k2^k)$ XOR butterflies and 2^k products of $O(W)$ -bit integers. Lane extraction and serialization add only linear passes with polynomial overhead. Together these give (3). Ordinary polynomial-time integer multiplication suffices for the simplified bound; no special machine-word assumption is needed. $\square$

Corollary 4.2 (Boundary-size bound). *For composition between matching objects on $w = 2m$ boundary points, the new dense algorithm has bit cost $\text{poly}(w)2^{w/2}$.*

Proof. Each of the three circle spaces is formed from two matchings and has at most m circles. Each gluing component contains at least one left input disc and at least one right input disc, so $k \leq \min(r, s) \leq m$. Apply Theorem 4.1. The crossing-free case is scalar arithmetic. $\square$

The reference support-pair loop can enumerate 2^{r+s} pairs even before its output arithmetic is counted. In a dense matching endomorphism algebra, $r = s = t = k = m$; its pair enumeration is of order 4^m , while the new dense bit algorithm is $\text{poly}(m)2^m$. This compares actual encoded arithmetic, not a fictitious unit-cost operation on exponentially long integers.

Remark 4.3 (Budgeted code versus the unbudgeted theorem). The shipped adapter has an allocation ceiling and a sparse fallback. When the ceiling excludes dense tables it deliberately uses the existing exact algorithm. That capped adapter does not inherit Corollary 4.2 on arbitrarily large widths. The theorem concerns the dense algorithm with sufficient memory; the budget policy concerns practical safety. An entry-count cap is also not a byte-count cap.

5 Optimal information compression and explicit planar families

5.1 The exact observational quotient

Definition 5.1. For a fixed plan, two left inputs are observationally equivalent if

$$f \sim_L f' \iff \text{Comp}(f, g) = \text{Comp}(f', g) \text{ for every right input } g.$$

Theorem 5.2 (Universal-summary optimality). *Assume that all components have genus zero and no extra dots, and that every L_j and R_j is nonempty. Then*

$$f \sim_L f' \iff P_L(f) = P_L(f').$$

There are exactly 2^{2^k} left observational equivalence classes. Every linear summary sufficient for all right-input composition queries has rank at least 2^k , and the component quotient attains this rank. Every fixed-length deterministic summary sufficient for those queries, even a nonlinear one, requires at least 2^k bits. The same conclusions hold with left and right exchanged.

Proof. The forward sufficiency of equal projections follows from Theorem 3.1. For necessity, let $a = P_L(f - f') \neq 0$. Both projections are surjective: choose one input circle on each component and map the corresponding square-free monomials onto the basis of $\mathcal{R}_k$.

Let λ be the coefficient functional selecting the monomial with *all* output circles dotted. When there are no output circles this means the scalar coefficient. By Lemma 3.2,

$$\lambda\Omega(h) = [y^{[k]}]h.$$

The bilinear pairing $(a, b) \mapsto [y^{[k]}]ab$ on $\mathcal{R}_k$ is nondegenerate: y^S and $y^{[k] \setminus S}$ are dual basis vectors. Select S with $a_S = 1$ and take $b = y^{[k] \setminus S}$. Surjectivity supplies a right input g with $P_R(g) = b$. Then

$$\lambda(\text{Comp}(f, g) - \text{Comp}(f', g)) = 1,$$

so the two left inputs are distinguishable.

The quotient is therefore the 2^k -dimensional space $\mathcal{R}_k$ itself, with 2^{2^k} elements. A sufficient linear summary has kernel contained in $\ker P_L$, giving the rank bound. An arbitrary sufficient deterministic summary must have distinct values on all these classes, giving the fixed-length information bound. The symmetric argument handles the right. $\square$

This is not a lower bound for all methods of deciding whether a knot is trivial. It is a lower bound for a precisely specified universal composition-summary interface on arbitrary coefficient vectors. It also does not rule out shorter descriptions of special input families.

Corollary 5.3 (Forced-dot refinement). *Under the same nonempty-input assumptions, suppose the extra-dot set is E and no component has two extra dots. Let q_E set $y_j = 0$ for $j \in E$. The exact left observational quotient is $q_E P_L$, of dimension $2^{k-|E|}$.*

Proof. For any $a, b \in \mathcal{R}_k$, $y^E ab = y^E q_E(a) q_E(b)$. The top-output coefficient now pairs the residual algebra on $[k] \setminus E$ by its top monomial. The proof of Theorem 5.2 applies to that residual algebra. $\square$

The code handles extra dots correctly, but does not yet use this smaller convolution dimension. Extra dots are absent in ordinary matching composition; the refinement is relevant to more general compiled maps.

5.2 Planar zipper triples

An arithmetic benchmark should not hide an impossible topology. We now give an explicit family of genuine planar matching triples.

Proposition 5.4 (Zipper family). *For any positive integers d, q , there are three noncrossing matchings a, b, c on $2q(2d - 1)$ commonly ordered boundary points such that the composition plan has*

$$r = s = qd, \quad t = k = q,$$

with q genus-zero components. Each component has d left input circles, d right input circles, and one output circle.

Proof. Construct one block with $m = 2d - 1$ arcs. Let b pair consecutive points, so its arcs are $(0, 1), (2, 3), \dots, (2m - 2, 2m - 1)$. To obtain a , replace the consecutive arc pairs with starting indices $j = 0, 2, \dots, 2d - 4$ by

$$(2j, 2j + 3), \quad (2j + 1, 2j + 2).$$

To obtain c , make the same replacements starting at $j = 1, 3, \dots, 2d - 3$. Each replacement lies in its own four-point interval, so the resulting matchings are noncrossing.

Each replacement merges two circles of $a \cup \bar{b}$, or of $b \cup \bar{c}$, and the replacements are disjoint within each matching. Both circle counts are therefore $m - (d - 1) = d$. Make a bipartite graph whose vertices are these left and right circles and whose edges are the m arcs of b . The alternating pairings make this graph a path on $2d$ vertices. Gluing the corresponding $2d$ discs along its $2d - 1$ edges produces a connected disc. Hence the gluing has one component, genus zero, and one output boundary circle. Take q disjoint consecutive copies of this construction. $\square$

For fixed q , the component algebra has dimension 2^q even though the input spaces each have dimension 2^{qd} . The required input scan is still present: compression does not make reading a dense input free. The benchmark with $(d, q) = (5, 2)$ uses this construction on 36 boundary points. The exact matchings and masks are stored in `results/examples.json`, and 24 zipper triples are checked by the tests.

A useful consistency identity follows from Euler characteristic. If a matching composition has m glued arcs, k components, t output circles, and total genus G , then

$$r + s - m = 2k - 2G - t. \tag{4}$$

The left side counts input discs minus glued intervals. In the genus-zero case this gives $r + s + t - m = 2k$. The zipper tests check this identity as well as the explicit component masks.

6 Integration and the remaining multiplicity problem

6.1 What the adapter changes

`ComponentAlgebra` wraps a new scanner’s existing `Planar` instance. It delegates geometry, circle numbering, stage setup, transfer, and existing shape/result caches. A stage-local cache stores compiled component quotients. Only sufficiently dense composition calls use the new kernel. Small support products and excluded allocations use the original composition. No global monkey-patch is used.

The default policy first requires at least 256 support pairs. It then compares that count with a rough table-and-transform work estimate. This is a transparent heuristic, not a theorem predicting Python wall time. Compiling a table also costs time and memory; amortization depends on actual plan reuse. Source-level identity shortcuts and complete result cache hits occur before this numerical kernel and should remain there.

The check callback propagates the scanner’s existing budget or race exception. A dense-allocation failure means “use the exact old kernel,” not “this is a knot.” The wrapper is owned by one scanner, and hence by one race worker if the caller installs it there. Cross-process installation and the production command-line interface have not been changed.

6.2 A parameter ledger for complete scans

Let n be the number of scanned crossings, w the maximum number of boundary endpoints, and Q the maximum number of explicit objects present at a stage *before* cancellation. Counting only the objects left after elimination would miss the transfer peak.

Proposition 6.1 (Dense-primitive scanner envelope). *A conventional explicit scanning-and-cancellation algorithm can be implemented with bit cost*

$$n \operatorname{poly}(n, Q, w) 2^{O(w)}. \quad (5)$$

If composition uses Theorem 4.1 and linear transfer maps use dense coefficient reconstruction as described below, an implementation has the sharper envelope

$$nQ^3 \operatorname{poly}(n, w, \log(Q + 1)) 2^{w/2}. \quad (6)$$

These are parameterized implementation envelopes, not uniform bounds on Q or w for all knot diagrams.

Proof. A stage has at most Q cancellations. Cancelling a unit creates at most Q^2 source-target update pairs, each requiring a bounded number of compositions. For this existence bound, a pivot can be selected by scanning at most Q^2 entries per cancellation; an optimal Markowitz choice is not required. Explicit arrays give deterministic access with polynomial bit overhead and logarithmic identifier sizes. Geometry on at most w endpoints is polynomial. Thus $O(nQ^3)$ composition updates suffice as an envelope.

Adding one crossing has a bounded number of smoothing and delooping choices per old object, and at most polynomially many entries in Q . For a fixed linear genus-zero transfer map, aggregate input monomials by component masks in a balanced search tree with at most 2^r keys. For each of the 2^t output subsets, Lemma 3.2 identifies at most one key to query. With no second input to

convolve, this takes $\text{poly}(w)(2^r + 2^t)$ bit operations. Crossing transfer has only a bounded number of such maps; untouched circles are included as one-input, one-output components. Here $r, t \leq w/2$ at the adjacent frontiers. Together with Corollary 4.2 this proves (6).

Even direct support expansions and pair evaluation have only $2^{O(w)}$ terms and $2^{O(w)}$ -bit intermediate morphisms. Combining these with the polynomial number of object updates gives (5). $\square$

The shipped adapter leaves the existing transfer code unchanged. Therefore (6) is an envelope for the stated dense-primitive implementation, not a measured or fully implemented bound for that adapter. The weaker exponential-in- w envelope is sufficient for the following criterion.

Corollary 6.2 (A sufficient scanning criterion). *Suppose a family has polynomial-size input encodings and computable scan schedules whose construction costs $n^{O(\log n)}$. If the schedules satisfy*

$$w + \log_2(Q + 1) = O((\log n)^2),$$

then the parameterized implementation in (5) computes its scans in $n^{O(\log n)}$ time.

Proof. Take binary logarithms of (5). A fixed polynomial in n, Q, w contributes $O(\log n + \log(Q + 1) + \log(w + 1))$, and the exponential factor contributes $O(w)$. The assumed schedule cost has the same quasi-polynomial upper bound. $\square$

Let M be the number of distinct matching types at a stage, and let μ be the maximum number of copies of any one matching, summed over all homological degrees. Then $Q \leq M\mu$. If all matchings are noncrossing in one certified cyclic order on the boundary of a disc, $M \leq \text{Cat}_{w/2} \leq 2^w$. Without that geometric hypothesis, the safe purely combinatorial bound for pairings is $M \leq (w - 1)!!$, giving

$$\log Q = O(w \log(w + 2)) + \log(\mu + 1).$$

A frontier-size argument must identify which setting it uses. We have not audited all repository scan frontiers to upgrade an observed matching count to a universal Catalan-bound theorem.

6.3 Fixed boundary does not control explicit multiplicity

Proposition 6.3 (Connected-sum multiplicity obstruction). *Choose a nontrivial knot K_0 and write $\beta = \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K_0; \mathbb{F}_2) > 1$. There is a family of $(1, 1)$ tangles T_q with two boundary endpoints, whose reduced closures have Khovanov dimension β^q . Any explicit complex of copies of the single boundary matching that is chain-homotopy equivalent to the tangle complex of T_q must contain at least β^q objects in total.*

Proof. Cut a marked strand in the connected sum of q copies of K_0 to obtain T_q . At the cube-of-resolutions level, the complex of a connected sum is the tensor product over the marked-circle algebra A . Each resolution is free over that marked factor. Applying the reduced base change $A \rightarrow A/(x) = \mathbb{F}_2$ identifies the reduced connected-sum complex with the tensor product over $\mathbb{F}_2$ of the reduced factor complexes. The field Künneth formula gives reduced homology dimension β^q .

Under reduced closure, each copy of the one-arc boundary matching yields one basis vector. Applying this additive functor to a chain homotopy equivalence preserves that equivalence. Homology dimension cannot exceed total chain-group dimension, so at least β^q objects are necessary. Finally, $\beta > 1$ follows from the rational reduced unknot-detection theorem [5] and the universal-coefficient inequality $\dim_{\mathbb{F}_2} \widetilde{\text{Kh}} \geq \dim_{\mathbb{Q}} \widetilde{\text{Kh}}$. $\square$

The crossing count of this family is linear in q for a fixed diagram of K_0 . Thus no polynomial multiplicity bound can depend on the two-point boundary alone. This is not a hardness proof for unknot recognition: the family consists of visibly composite nontrivial knots, and the repository's existing connected-sum stage is designed to avoid such redundant full computations. The point is narrower and useful: an unrestricted proof may not count boundary matchings and silently discard their multiplicities.

7 Sparse hierarchy restart entropy

7.1 The source of the lexicographic profile

Lackenby's 2026 preprint uses integer pattern-complexity $\chi_p(S) = -4\chi(S) + |S \cap P|$. In Section 9, nonnegative layer complexities are padded to length L ; between successive construction-loop exits, a first changed coordinate decreases and its suffix is rebuilt. The resulting bound is $L(g+1)^L$ iterations when layer values are at most g . This counts iterations, not their bit costs [6, Section 9].

The refinement below is a theorem about that restart pattern. Its new hypothesis is a bound on the number of positive coordinates in every relevant profile. No assertion that all knot exteriors satisfy this hypothesis is made.

Definition 7.1 (Sparse profile space). For nonnegative integers L, g, s , set

$$\mathcal{V}(L, g, s) = \{x \in \{0, 1, \dots, g\}^L : |\{i : x_i > 0\}| \leq s\}.$$

A restart trace is a sequence of these vectors in strictly decreasing lexicographic order. One may replace all coordinates after the first changed coordinate arbitrarily, subject to membership in $\mathcal{V}$.

The actual positions of zero-complexity layers are not erased. They can carry topology and affect where positive layers occur. This is why the count below contains $\binom{L}{j}$, rather than merely g^s .

Theorem 7.2 (Sharp sparse restart bound). *The profile space has cardinality*

$$N(L, g, s) = \sum_{j=0}^{\min\{s, L\}} \binom{L}{j} g^j. \quad (7)$$

A strictly descending trace has at most $N(L, g, s)$ states and at most $N(L, g, s) - 1$ restart transitions. If an epoch between restarts uses at most $C(L+1)$ primitive calls, the entire trace and its final epoch use at most $C(L+1)N(L, g, s)$ calls. The state bound is sharp under the stated abstract restart rules.

Proof. To choose a vector with exactly j positive coordinates, choose their locations in $\binom{L}{j}$ ways and assign each of them one of g positive values. Summing gives (7), including the all-zero vector. Strict descent cannot revisit a vector, so it has the stated length. Multiplying the number of epochs by their assumed call bound proves the algorithmic count.

For sharpness, list every vector of $\mathcal{V}$ in decreasing lexicographic order. Between consecutive vectors, the first differing coordinate decreases; all earlier coordinates agree. The later coordinates are a permissible replacement suffix. Hence this full list is a legal abstract restart trace with exactly N states. $\square$

Corollary 7.3 (Polynomial total depth, logarithmic positive depth). *For $n \geq 2$, if $L, g \leq n^a$ and $s \leq b \log_2 n$ for fixed constants a, b , then*

$$N(L, g, s) = n^{O(\log n)}.$$

The same is true after multiplying by a polynomial bound on the number of primitive calls per epoch.

Proof. For $Lg \geq 1$,

$$N(L, g, s) \leq (s + 1) \max\{1, Lg\}^s.$$

Taking logarithms gives $\log_2 N \leq \log_2(s + 1) + s(\log_2 L + \log_2 g) = O((\log n)^2)$. When $L = 0$ or $g = 0$, the space has one vector. $\square$

A mere polynomial bound on L is insufficient without another restriction: with $g = 1$ and unrestricted support, reverse binary counting gives 2^L states. Theorem 7.2 is therefore not a repackaging of “polynomial depth implies quasi-polynomial time.” It identifies precisely which additional combinatorial resource would suffice.

7.2 An exact executable rank potential

Let $\rho(x)$ be the number of members of $\mathcal{V}(L, g, s)$ strictly smaller than x in increasing lexicographic order. While scanning a prefix of x , suppose r nonzero positions remain available and there are m coordinates after the current one. If the current digit d is positive, its contribution to the rank is

$$N(m, g, r) + (d - 1)N(m, g, r - 1). \quad (8)$$

The first term counts the smaller choice zero. The second counts smaller positive digits. Continue with remaining budget $r - 1$. A zero current digit contributes nothing. This proves the rank recurrence used by `lex_rank`. Its inverse compares the rank with the zero-digit block size and then uses division by the positive-digit block size.

The potential is an exact nonnegative integer, decreases at every restart, and is initially at most $N - 1$. It uses only $\lceil \log_2 N \rceil$ bits. Evaluation and inverse evaluation use binomial coefficients and integer arithmetic polynomial in their explicit dimensions and bit lengths. The trace tool is not an oracle for topology: it checks a supplied finite trace and cannot prove that an unspecified algorithm will always preserve the sparsity bound.

For $L = 4$, $g = 2$, and $s = 2$, the count is $1 + 8 + 24 = 33$. The package supplies the complete reverse-ranked trace of these 33 vectors. It also checks 23,460 rank/unrank identities and 140 complete countdowns.

7.3 Other entropy budgets

Proposition 7.4 (Heterogeneous and additive budgets). *If coordinate i has a fixed cap g_i , the unrestricted state count is $\prod_i (g_i + 1)$. With support at most s , it is*

$$\sum_{j=0}^s [z^j] \prod_{i=1}^L (1 + g_i z).$$

If instead nonnegative coordinates satisfy $\sum_i x_i \leq G$, their state count is $\binom{L+G}{G}$, before imposing any further coordinate caps.

Proof. The first formula is the product rule. In the generating function, the constant term at a position selects zero, and $g_i z$ selects one of its positive values. For the additive budget, adjoin a nonnegative slack coordinate to make the total equal to G and apply stars and bars. $\square$

Thus another possible target is total charge $G = O(\log n)$ with polynomial L . Fixed-coordinate caps have entropy $\sum_i \log_2(g_i + 1)$. These alternatives can be useful when a maximum cap g is a poor summary. The caps must be valid uniformly over the trace, not chosen retrospectively only for the currently observed vector.

7.4 What would complete a hierarchy-based quasi-polynomial proof

Theorem 7.5 (Conditional hierarchy criterion). *Suppose an exact hierarchy-based recognition algorithm on inputs of size n has all of the following properties. Its restart profiles lie in $\mathcal{V}(L, g, s)$ and strictly decrease; L, g are polynomially bounded and $s = O(\log n)$; at most polynomially many primitive calls occur in an epoch; and every primitive has a uniform $n^{O(\log n)}$ bit-time bound, including construction and maintenance of its encoded state. Assume its initialization has the same bound and its terminal decisions are sound and complete. Then the algorithm recognizes the unknot in $n^{O(\log n)}$ time.*

Proof. Corollary 7.3 bounds the number of epochs by $2^{O((\log n)^2)}$. Multiplying this by polynomially many calls per epoch and by $2^{O((\log n)^2)}$ work per call preserves that form. Add the initialization cost and use the assumed decision correctness. $\square$

This is a sufficient-condition theorem, not a completed topological algorithm. The difficult missing statement is a *constructive*, uniform sparsity or comparable entropy bound for all states the algorithm actually reaches, including rebuilt suffixes and failed choices. An efficiently describable successful certificate alone does not supply that statement.

The source already proves important compressed ingredients: Section 10 controls refined hierarchy length linearly in initial q -complexity; Theorem 14.1 gives polynomial-size verifiable essential hierarchies; Proposition 14.2 gives polynomial-time cut-and-simplify operations under its hypotheses, measured in handle count and logarithmic extended surface weight [6]. These should be reused, not described as absent. What remains for our criterion is uniform efficient construction, valid hypotheses throughout the run, and low restart entropy. Zero potential increments do not make geometric operations free.

8 Validation and performance experiments

8.1 What was executed

The artifact is standard-library Python, tested here on Python 3.13.5, GCC 14.2.0, Linux x86_64. The final recorded run completed 15 test methods with zero failures and zero errors in 7.07 seconds. This is a numerical kernel and abstract-potential test suite, not the repository’s full suite.

Validation category	Recorded checks
Subset-convolution comparisons and square identities	65,980
Matching-composition equalities	460,564
General plans: counits, extra dots, and splitting	24,000
Associativity identities	1,000
Adaptive-adapter comparisons	31
Explicit zipper triples	24
Sparse-profile rank/unrank identities	23,460
Complete sparse countdown traces	140

All coefficient pairs are checked for all planar matching triples through six boundary points whenever their plan has genus zero. Larger randomized triples cover eight through fourteen endpoints. The 2,535 sampled or exhaustively enumerated triples include 590 positive-genus plans, which are counted separately rather than as successful dense-kernel evaluations. The general-plan tests cover output counits, zero maps, multiple extra dots, and empty universes. Invalid masks, out-of-basis coefficients, allocation limits, stage resets, and cooperative budget exceptions also have tests.

The reference is an excerpt of the repository’s actual numerical algorithms, not a second full topological implementation. Equality with it is useful regression evidence but could share a source convention error. The monomial proof of Theorem 3.1, the independent subset-convolution checks, and the associativity checks supply different kinds of evidence. None substitutes for a full scanner regression suite or a formal proof assistant verification.

8.2 Paired numerical-kernel measurements

The benchmark uses 11 rounds, seed 20261007, and alternating ABAB/BABA measurement order. Each round checks output equality and records two old and two new timings. A/A ratios are retained. Topology is precompiled for both implementations; component-table compilation is timed separately. The reference numerical monomial memo is fresh on each invocation. The new method has no numerical-result cache. This measures a particular cold-memo numerical workload, not the amortized behavior of a complete scanner with all its caches.

Case	Old (ms)	New (ms)	Paired speed ratio	Compile (ms)
Dense endomorphism, $k = 4$	0.0322	0.0206	1.56	0.0444
Dense endomorphism, $k = 6$	0.5877	0.1162	5.03	0.0908
Dense endomorphism, $k = 8$	9.8212	0.4247	22.89	0.2890
Dense endomorphism, $k = 10$	179.5399	2.1522	86.72	1.3189
Sparse endomorphism, $k = 10$	0.0126	1.2923	0.00972	1.3195
Zipper, $(r, s, t, k) = (10, 10, 2, 2)$	69.1873	0.1742	401.98	0.2521

Old and new columns are separate medians; the speed ratio is the inverse of the median paired new/old ratio, so it need not equal their quotient. The sparse case has four terms in each operand and deliberately forces the dense kernel. It is approximately 103 times slower by the paired median. The adapter’s default 256-pair gate avoids that particular substitution.

The observed paired new/old ranges for dense $k = 6, 8, 10$ are respectively $[0.142, 0.234]$, $[0.0336, 0.0540]$, and $[0.0105, 0.0128]$. The zipper range is $[0.00201, 0.00267]$. These are ranges of observed rounds, not confidence intervals. The $k = 4$ range extends above one, so its small

median gain is not a robust universal speed claim. The A/A controls and every raw timing are available in `results/benchmark.json`.

These are genuine matching-algebra workloads, including the explicit planar zipper family, but they are *not measured recognition speedups on knot diagrams*. Existing pipeline filters may settle an input before Khovanov computation; existing caches may remove numerical work; and many scanner operands may remain sparse. Full-input paired timing, warm-memo behavior, peak memory, and the frequency of compressed calls are necessary before enabling the adapter by default.

8.3 Reproduction and repository validation

From the package root:

```
python tools/run_tests.py
python tools/benchmark.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The `Makefile` provides the corresponding targets. Timing values will vary by machine and run; structural counts should be reproducible. The article’s table records the delivered run, rather than automatically changing when somebody reruns the benchmark.

`integration/check_repository.py` is a read-only local-checkout harness. It reports audited source hashes, compares the kernel against the actual `Planar`, and optionally compares direct closed-PD scanner runs with $d^2 = 0$ checks. It was supplied but not executed against a complete checkout in this environment. No Rust port, full repository suite, or production recognizer benchmark is claimed. The integration guide states these limits and the required pre-adoption checks explicitly.

9 Further research questions and completion criteria

9.1 Production relevance and adaptive arithmetic

Research question 9.1. On the current hard-input corpus, how often does composition reach the dense regime after the existing filters, cancellation, identity shortcuts, and caches have run?

A useful result is a paired full-scanner study with compressed-call counts, operand densities, component counts, table-reuse counts, and peak memory. The decision criterion should be improvement in completed recognition or scan time, not merely a large numerical-kernel ratio. Preserve cases with zero compressed calls in the report. An implementation milestone is default enablement only after complete Python cross-checks and a separate Rust port validation.

Research question 9.2. Can a dispatcher choose between sparse pair evaluation, component projection, and packed convolution with bounded overhead relative to the best available arithmetic strategy?

The current support threshold is heuristic. A next version should measure warm-memo reuse and compilation amortization and should include the forced-dot refinement of Corollary 5.3. A theoretical target is a transparent online work bound under a declared primitive-cost model; a practical target is paired speed with no sparse-input regression beyond a specified tolerance. Dense-linear transfer reconstruction is another well-defined implementation task needed for the sharper scanner envelope.

9.2 Representations that do not enumerate multiplicity

Research question 9.3. For which tangle families does a tensor, circuit, or module presentation retain the cancellation information needed for an unknot decision while avoiding explicit copies of every matching object?

Proposition 6.3 excludes a width-only bound for the explicit representation, not for every representation. A successful theorem must specify how composition, testing units, cancellation, and the final decision operate on the compressed presentation. Short descriptions of chain groups alone do not bound the cost of deciding their homology. First targets could be families closed under a specified tangle composition with proved bounds on description size and update cost.

Research question 9.4. Can one maintain a decision-sufficient certificate instead of computing the entire reduced Khovanov rank on difficult nontrivial inputs?

The universal-summary theorem concerns all opposite-operand composition queries, a much stronger demand than one final decision. A useful advance would identify a smaller query class justified by the scanner and prove that its retained information is sound under every continuation. A shortcut must not interpret failure to discover a second homology class as proof of rank one. Existing connected-sum reduction should remain part of this investigation, rather than being counted as a new result.

9.3 Low-entropy hierarchy construction

Research question 9.5. Is there a constructive hierarchy strategy for knot exteriors for which every restart profile has at most $C \log n$ positive pattern-complexity layers, with polynomial coordinate caps and total padded length?

The required bound must survive unsuccessful surface choices, suffix reconstruction, and simplification. A theorem only about a best final certificate is insufficient. Theorem 7.5 gives a precise completion criterion: prove this uniform sparsity and the required primitive bit costs, or replace sparsity by another profile entropy bound of order $(\log n)^2$.

Research question 9.6. Can zero-complexity layers be represented by composable geometric records whose replay, transport of patterns, and invalidation after a restart have polynomial total cost?

Zero pattern-complexity does not mean a layer is an identity operation. The representation must preserve boundary attachments and certificates that local replacements induce the intended manifold operations. A promising target is a persistent directed acyclic record of cuts and parallelity reductions with explicit size, composition, and verification bounds. A necessary audit separates constructing a useful surface from cutting along a surface that has already been supplied.

Research question 9.7. Which large abstract restart countdowns are realizable by actual hierarchies and a fixed deterministic choice strategy?

The sharpness construction in Theorem 7.2 is combinatorial, not a topological lower bound. Explicit diagram families, including prime and iterated satellite families, could distinguish genuine geometric constraints from artifacts of an overpermissive state model. Conversely, a proved family with many positive layers would refute the naive uniform sparsity conjecture for that particular strategy and motivate heterogeneous or additive charge budgets.

9.4 Verification and scan scheduling

Research question 9.8. Can the component factorization, its observational quotient, and the sparse-rank potential be formalized with extraction of the executable arithmetic, then connected to verified geometric primitives?

The finite-set algebra offers a relatively small first formalization: quotient-ring maps, the top-coefficient pairing, and ranked Möbius inversion. The restart theorem requires finite lexicographic order and exact counting. The geometric interface is a separate proof obligation: compiled masks must describe the intended surface, and recorded hierarchy transitions must preserve their topological invariants. Regression tests should continue to target those interfaces.

A related scheduling task is to predict work from both boundary width and object multiplicity. The profile (w, Q, k) and observed morphism density are more informative than width alone. Any new schedule search must count its own running time and must be compared with the repository's existing order selection and racing rather than treated as free preprocessing.

10 Conclusion

The immediate implementable advance is an exact dense-arithmetic kernel for the existing scanning backend. Its governing variable is the number of connected gluing components, and its composition cost on w endpoints is $\text{poly}(w)2^{w/2}$ in the dense bit model. The exact observational quotient and the planar zipper family explain both why the compression is valid and where substantial reductions can occur. Sparse workloads, memory caps, transfer costs, and multiplicity remain visible in the analysis.

For the global asymptotic objective, sparse restart entropy is a distinct and concrete target. Polynomial hierarchy length is compatible with quasi-polynomial restart counts when only logarithmically many coordinates are positive. What must still be proved is a constructive topological bound of that kind, together with complete primitive cost accounting. The supplied proofs, code, negative cases, test records, and integration harness are intended to make those next steps experimentally accessible without overstating what has already been established.

A Artifact map and audit boundaries

Path	Role
article.tex, article.pdf	Self-contained article source and compiled text.
code/component_kernel.py	Exact quotient, convolution, reconstruction, and opt-in adapter.
code/restart_entropy.py	Counts, lexicographic ranks, inverse ranks, finite-trace audit.
tests/reference_planar.py	Numerical excerpt from the audited repository source; not a full recognizer.
tests/fixtures.py	Explicit planar zipper construction.
tests/test_*.py	Regression and structural checks.
tools/run_tests.py,	Reproducible validation and paired kernel measurements.
tools/benchmark.py	
results/	Raw JSON, console records, worked examples, and finite traces.
integration/	Opt-in usage and unexecuted local-checkout validation harness.
PROVENANCE.json, CLAIMS.md	Exact source identifiers and claim/limitation ledger.

No third-party lecture slides or research-paper PDFs are redistributed. The source excerpt comes from the repository’s MIT-0 material. The package contains no automatic modification of the production repository. The root README and claim ledger distinguish delivered execution results from suggested future validation commands.

References

- [1] V. Reshetnikov and contributors, *ProveIt: Topology/UnknotRecognition*, source audited 7 October 2026. <https://github.com/VladimirReshetnikov/ProveIt/tree/main/Topology/UnknotRecognition>. Exact reviewed blob identifiers are recorded in PROVENANCE.json.
- [2] D. Bar-Natan, *Khovanov’s homology for tangles and cobordisms*, arXiv:math/0410495. <https://arxiv.org/abs/math/0410495>.
- [3] D. Bar-Natan, *Fast Khovanov Homology Computations*, arXiv:math/0606318. <https://arxiv.org/abs/math/0606318>.
- [4] A. Björklund, T. Husfeldt, P. Kaski, and M. Koivisto, *Fourier meets Möbius: fast subset convolution*, arXiv:cs/0611101 (2006), Proceedings of STOC 2007. <https://arxiv.org/abs/cs/0611101>.
- [5] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, arXiv:1005.4346; Publications Mathématiques de l’IHÉS **113** (2011), 97–208. <https://arxiv.org/abs/1005.4346>.
- [6] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. <https://people.maths.ox.ac.uk/lackenby/algorithm-incompressible-250726.pdf>.